

# Rapport individuel

**PUPP-BAUDSON Gwendan**

**Groupe 14**

**SAE 4.01a**

## I. Tâches réalisées

### 1) Mise en place API REST

Pour cette partie de la SAE, j'ai d'abord mis en place l'API REST en m'appuyant du MCD qui a été modifié exprès pour cette partie. J'ai tout d'abord créé tous les modèles avec SQLAlchemy, et j'ai alimenté tout le jeu de données qui est utilisé dans la commande flask syncdb. En m'inspirant de la ressource d'architecture logicielle, j'ai alors pu établir toutes les routes CRUD sur chaque modèle de la base de données. À la fin, j'ai pu réaliser les tests avec coverage de toutes les vues et de tous les modèles.

### 2) Base et méthode GET pour la SPA

Dès que j'ai fini l'API REST, j'ai voulu faire la base de la SPA. Pour cela, j'ai recherché dans les différentes documentations une base propice à une SPA de ce type. Je me suis alors tourné vers un dashboard tiré d'un exemple de bootstrap (<https://getbootstrap.com/docs/5.3/examples/dashboard/>). J'ai choisi cette base, car je trouvais intéressant l'idée de pouvoir naviguer à gauche sur toutes les fonctionnalités, et que sur la partie de droite, on puisse afficher des formulaires ou des cartes sans devoir forcer pour que ça prenne trop d'espace. Après avoir adapté l'exemple pour qu'elle concorde à nos besoins (nom des liens, les images, l'arborescence), j'ai effectué les requêtes GET de chaque instance, en donnant la possibilité d'accéder au détail de l'instance choisie en cliquant dessus. J'ai ensuite mis en place un système de filtre avec une barre de recherche sur chaque page, et trié selon le nom (de l'aéroport, de la compagnie, du vol ou du terminal). J'ai dû apporter une modification dans l'API pour prendre en compte des requêtes paramétrées. Pour finir, j'ai fait en sorte d'avoir la pagination, pour que sur une page, on affiche au plus 10 instances. J'ai eu d'importantes difficultés à mettre en place la pagination, car je n'avais pas pensé à configurer l'API pour prendre en compte les routes avec des options telles que `per_page`. J'ai alors dû faire la pagination en front-end, en manipulant l'affichage du render pour jouer avec la pagination.

### 3) README et suivi de projet

Pour finir, j'ai d'abord complété le README de façon à expliquer ce qu'il faut installer et comment, pour pouvoir lancer l'API REST et la SPA. J'ai ensuite rédigé le document de suivi de projet en m'appuyant sur le Gantt qu'on a mis en place et modifié au fur et à mesure de la SAE. Dans ce document, j'ai rappelé le contexte de cette partie de SAE, le déroulé de ce projet en respectant l'ordre chronologique, et j'ai rappelé le rôle de chacun des membres du groupe dans la SAE.

## II. *Démonstration de compétences*

Pour cette partie de SAE, j'ai créé une API REST complète en m'appuyant sur le MCD modifié pour cette partie. J'ai traduit les différentes exigences à travers les modèles avec SQLAlchemy et les routes CRUD (cf Annexe 1 à 5), ce qui répond aux besoins fonctionnels définis et montre que j'ai su mettre en œuvre l'apprentissage critique AC21.01. En utilisant une architecture en couches, SQLAlchemy pour la modélisation (cf Annexe 9), et en ayant réalisé des tests, j'ai adopté de bonnes pratiques de conception et de programmation (AC21.03).

De plus, j'ai fait le choix d'un dashboard tiré de Bootstrap pour avoir une ergonomie pensée, par rapport à la navigation latérale, affichage plus flexible des formulaires et des cartes et avec la pagination (cf Annexe 7 et 8). Cela montre que j'ai veillé à appliquer des principes d'accessibilité et d'ergonomie (AC21.02).

J'ai également réalisé des tests avec coverage sur toutes les vues et tous les modèles de l'API REST, avec un taux de couverture total à 98 % comme vous pouvez le voir sur l'annexe 6. Cela montre que j'ai vérifié et validé la qualité de l'application par les tests (AC21.04).

D'autre part, en participant à la mise en place une SPA en faisant l'affichage des données sous forme de cartes (cf Annexe 7), des détails d'instances accessibles en un clic (cf Annexe 8), de l'option de filtrage et la pagination, j'ai effectué une restitution organisée des données de la base de données (AC24.03). En gérant plusieurs modèles différents (aéroports, terminaux, vols, compagnies) avec pour chacun leurs propres routes CRUD, et avec des requêtes paramétrées, j'ai manipulé des données hétérogènes (AC24.04).

L'implémentation d'un système de filtrage qui trie les données (par nom d'aéroport, de terminal, de vol ou de compagnie) ainsi que de la pagination,

montre que j'ai utilisé des techniques algorithmiques adaptées pour des problèmes complexes (AC22.02).

Finalement, j'ai développé une API REST complète, et qui communique avec une SPA (et avec l'application mobile, mais je n'y ai pas participé). Les requêtes GET paramétrées permettent à la SPA de dialoguer dynamiquement avec le serveur. Cela prouve que j'ai bien su concevoir et développer des applications communicantes (AC23.01).

J'ai aussi rédigé le document de suivi de projet basé sur le diagramme de Gantt. J'y ai décrit le contexte de cette partie Web/Mobile, expliqué chronologiquement les tâches qu'ont été réalisées et définis les rôles au sein du groupe. Cela montre que j'ai défini et mis en œuvre une démarche de suivi de projet (AC25.04). Pour finir, j'ai expliqué dans le fichier README comment installer et lancer le projet. De ce fait, additionné avec la rédaction du suivi de projet, cela montre que j'ai rendu compte de mon activité professionnelle (AC26.04).

# Annexe

## api

|        |                                        |                                                                       |   |
|--------|----------------------------------------|-----------------------------------------------------------------------|---|
| POST   | /api/aeroports                         | Crée un nouvel aéroport                                               | ⌵ |
| GET    | /api/aeroports                         | Récupère la liste des aéroports, avec possibilité de filtrer par nom  | ⌵ |
| PUT    | /api/aeroports/{id_aeroport}           | Modifie un aéroport existant                                          | ⌵ |
| DELETE | /api/aeroports/{id_aeroport}           | Supprime un aéroport par son ID                                       | ⌵ |
| GET    | /api/aeroports/{id_aeroport}           | Récupère un aéroport par son ID                                       | ⌵ |
| GET    | /api/aeroports/{id_aeroport}/terminaux | Récupère les terminaux d'un aéroport par son ID                       | ⌵ |
| POST   | /api/compagnies                        | Crée une nouvelle compagnie                                           | ⌵ |
| GET    | /api/compagnies                        | Récupère la liste des compagnies, avec possibilité de filtrer par nom | ⌵ |
| PUT    | /api/compagnies/{id_compagnie}         | Modifie une compagnie existante                                       | ⌵ |
| DELETE | /api/compagnies/{id_compagnie}         | Supprime une compagnie par son ID                                     | ⌵ |
| GET    | /api/compagnies/{id_compagnie}         | Récupère une compagnie par son ID                                     | ⌵ |
| GET    | /api/compagnies/{id_compagnie}/vols    | Récupère les vols d'une compagnie par son ID                          | ⌵ |
| POST   | /api/localisations                     | Crée une nouvelle localisation                                        | ⌵ |
| GET    | /api/localisations                     | Récupère la liste des localisations                                   | ⌵ |
| POST   | /api/terminaux                         | Crée un nouveau terminal                                              | ⌵ |

*Annexe 1 : Extrait de toutes les routes de l'API REST*

GET

/api/aeroports

Récupère la liste des aéroports, avec possibilité de filtrer par nom

Parameters

Cancel

| Name                                   | Description             |
|----------------------------------------|-------------------------|
| X-Fields                               | An optional fields mask |
| <div>string(\$mask)<br/>(header)</div> | <div>X-Fields</div>     |

Execute

Clear

Responses

Response content type

application/json

Curl

```
curl -X 'GET' \
'http://127.0.0.1:5000/api/aeroports' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:5000/api/aeroports
```

Server response

| Code | Details                                                                                                                                                                                                                                                                                                                                                                                                                |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <div>Response body</div> <div><pre>[   {     "id_aeroport": 1,     "nom_aeroport": "Charles de Gaulle",     "ville": "Roissy",     "pays": "France",     "uri": "http://127.0.0.1:5000/api/aeroports/1"   },   {     "id_aeroport": 2,     "nom_aeroport": "Aéroport de Londres-Heathrow",     "ville": "Hounslow",     "pays": "Royaume-Uni",     "uri": "http://127.0.0.1:5000/api/aeroports/2"   },   {</pre></div> |

Annexe 2 : Exemple résultat route GET

POST

/api/terminaux

Crée un nouveau terminal

⌵

Parameters

Cancel

| Name                                                                                   | Description                                                                                                                                                                                                             |
|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>payload</div><div>★ required</div></div> <div>object</div> <div>(body)</div> | <div><div>Edit Value</div><div>Model</div></div> <div><pre>{   "nom_terminal": "Terminal 5",   "id_aeroport": 1 }</pre></div> <div>Cancel</div> <div><div>Parameter content type</div><div>application/json</div></div> |

Execute

Clear

Responses

Response content type

application/json

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:5000/api/terminaux' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "nom_terminal": "Terminal 5",
    "id_aeroport": 1
  }'
```

Request URL

http://127.0.0.1:5000/api/terminaux

Server response

| Code | Details                                        |
|------|------------------------------------------------|
| 201  | <div><div>Response body</div><div></div></div> |

Undocumented

Annexe 3 : Exemple résultat route POST

PUT

/api/compagnies/{id\_compagnie}

Modifie une compagnie existante

Parameters

Cancel

| Name                                                                                          | Description                                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>payload <span>★ required</span></div><div>object</div><div>(body)</div></div>       | <div><div>Edit ValueModel</div><div><div>{<br/>  "nom_comp": "Airbus"<br/>}</div></div></div> <div><div>Cancel</div><div>Parameter content type</div><div>application/json</div></div> |
| <div><div>X-Fields</div><div>string(\$mask)</div><div>(header)</div></div>                    | <div><div>An optional fields mask</div><div>X-Fields</div></div>                                                                                                                       |
| <div><div>id_compagnie <span>★ required</span></div><div>integer</div><div>(path)</div></div> | <div>1</div>                                                                                                                                                                           |

Execute

Clear

Responses

Response content type

application/json

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:5000/api/compagnies/1' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "nom_comp": "Airbus"
  }'
```

Request URL

http://127.0.0.1:5000/api/compagnies/1

Server response

CodeDetails

200

Response body

```
{
  "id_compagnie": 1,
  "nom_comp": "Airbus",
  "uri": "http://127.0.0.1:5000/api/compagnies/1"
}
```

Download

Annexe 4 : Exemple résultat route PUT

**DELETE**
/api/aeroports/{id\_aeroport}
 Supprime un aéroport par son ID

Parameters

Name

Description

id\_aeroport required

integer

(path)

1

Execute

Clear

Responses

Response content type application/json

Curl

```
curl -X 'DELETE' \
'http://127.0.0.1:5000/api/aeroports/1' \
-H 'accept: application/json'
```

Request URL

http://127.0.0.1:5000/api/aeroports/1

Server response

Code

Details

204

Undocumented

Response headers

```
access-control-allow-origin: http://127.0.0.1:5000
connection: close
content-type: application/json
date: Fri, 27 Mar 2026 14:59:25 GMT
server: Werkzeug/3.1.5 Python/3.12.3
vary: Origin
```

Annexe 5 : Exemple résultat d'une route DELETE

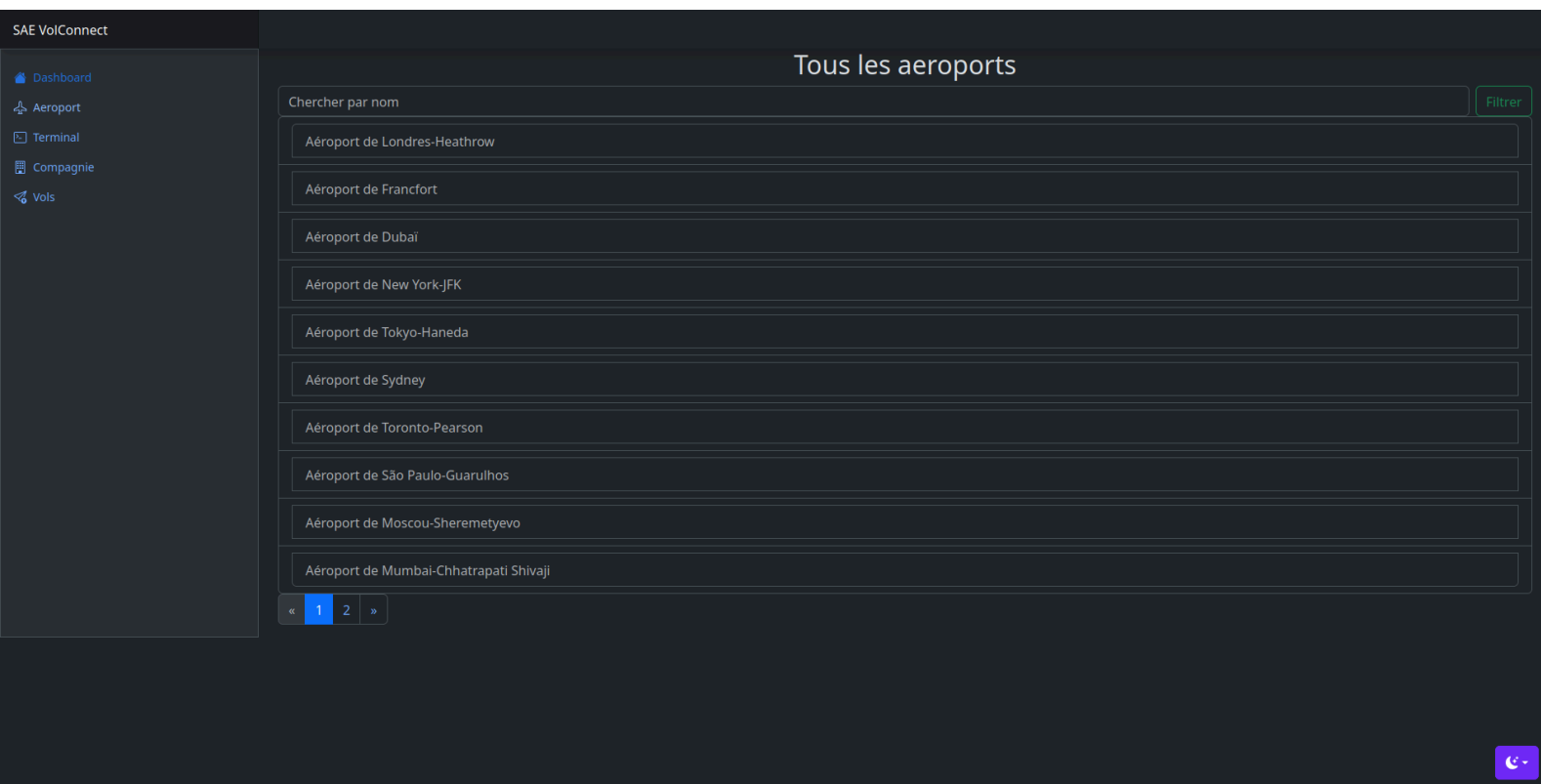
```

• (venv) addra@VivoBook-ASUSLaptop:~/Documents/BUT2/SAE/SAE_VolConnect/vol$ coverage report -m

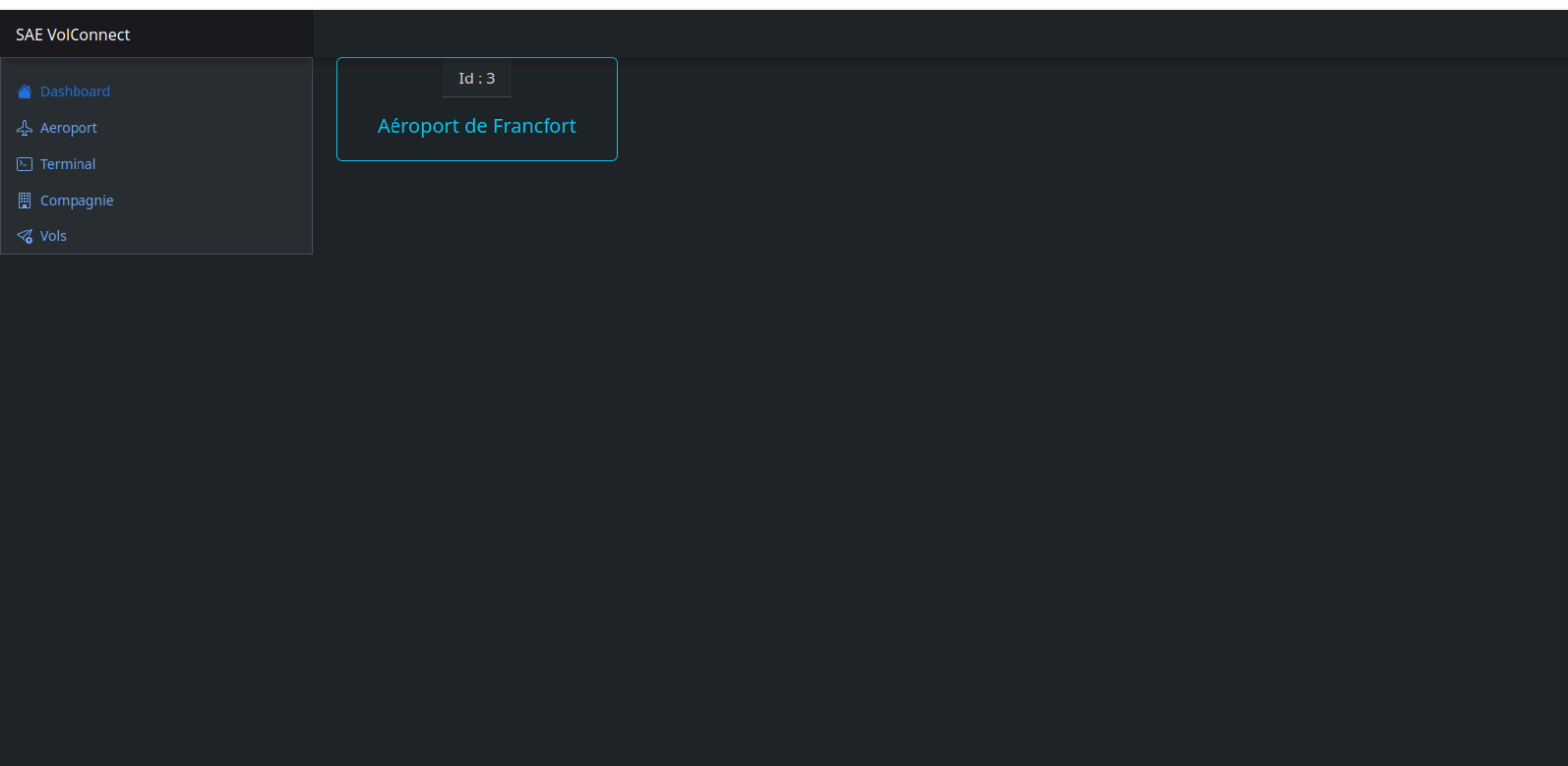
```

| Name                                               | Stmts | Miss | Cover | Missing                   |
|----------------------------------------------------|-------|------|-------|---------------------------|
| app/ __init__.py                                   | 2     | 0    | 100%  |                           |
| app/api_models.py                                  | 14    | 0    | 100%  |                           |
| app/extensions.py                                  | 4     | 0    | 100%  |                           |
| app/models.py                                      | 147   | 0    | 100%  |                           |
| app/myapp.py                                       | 10    | 0    | 100%  |                           |
| app/tests/conftest.py                              | 54    | 6    | 89%   | 39-40, 84-87              |
| app/tests/functionnal/test_routes_aeroports.py     | 29    | 0    | 100%  |                           |
| app/tests/functionnal/test_routes_compagnies.py    | 28    | 0    | 100%  |                           |
| app/tests/functionnal/test_routes_localisations.py | 6     | 0    | 100%  |                           |
| app/tests/functionnal/test_routes_terminaux.py     | 25    | 0    | 100%  |                           |
| app/tests/functionnal/test_routes_vols.py          | 26    | 0    | 100%  |                           |
| app/views.py                                       | 156   | 6    | 96%   | 15, 59, 70, 114, 140, 183 |
| TOTAL                                              | 501   | 12   | 98%   |                           |

Annexe 6 : Résultat des tests coverage



*Annexe 7 : Affichage de tous les aéroports dans la SPA*



*Annexe 8 : Affichage de détails d'un aéroport*

```

models.py X
vol > app > models.py > modify_terminal
1  from .extensions import db
2  from datetime import datetime
3
4  class Compagnie(db.Model):
5      """Modèle de données pour une compagnie aérienne."""
6      __tablename__ = 'Compagnie'
7      id_compagnie = db.Column(db.Integer, primary_key=True)
8      nom_comp = db.Column(db.String(100))
9
10 class Aeroport(db.Model):
11     """Modèle de données pour un aéroport."""
12     __tablename__ = 'Aeroport'
13     id_aeroport = db.Column(db.Integer, primary_key=True)
14     nom_aeroport = db.Column(db.String(100))
15     ville = db.Column(db.String(100))
16     pays = db.Column(db.String(100))
17
18 class Localiser(db.Model):
19     """Modèle de données pour une localisation. La table relie les compagnies aériennes aux aéroports où elles opèrent."""
20     __tablename__ = 'Localiser'
21     id_aeroport = db.Column(db.Integer, db.ForeignKey('Aeroport.id_aeroport'), primary_key=True)
22     id_compagnie = db.Column(db.Integer, db.ForeignKey('Compagnie.id_compagnie'), primary_key=True)
23     compagnie = db.relationship('Compagnie', backref=db.backref('Localiser', lazy="dynamic", cascade="all, delete-orphan"))
24     aeroport = db.relationship('Aeroport', backref=db.backref('Localiser', lazy="dynamic", cascade="all, delete-orphan"))
25
26 class Terminal(db.Model):
27     """Modèle de données pour un terminal."""
28     __tablename__ = 'Terminal'
29     id_terminal = db.Column(db.Integer, primary_key=True)
30     nom_terminal = db.Column(db.String(100))
31     id_aeroport = db.Column(db.ForeignKey('Aeroport.id_aeroport'))
32     aeroport = db.relationship('Aeroport', backref=db.backref('Terminal', lazy="dynamic", cascade="all, delete-orphan"))
33

```

## Annexe 9 : Exemple de modèles avec SQLAlchemy